

Datentyp `long` in Java

Thema	Beschreibung
Typ	<code>long</code> (primitive data type)
Größe	64 Bit (8 Byte)
Wertebereich	<code>-9.223.372.036.854.775.808</code> bis <code>9.223.372.036.854.775.807</code> (<code>-2⁶³</code> bis <code>2⁶³ - 1</code>)
Standardwert	<code>0L</code>
Wrapper-Klasse	<code>Long</code> (<code>java.lang.Long</code>)
Speicherung	Zweierkomplement-Darstellung
Explizite Umwandlung (Casting)	<code>(long) doublewert</code> - Umwandlung von <code>double</code> zu <code>long</code> , Abrundung erfolgt
Implizite Umwandlung	<code>long</code> kann automatisch in <code>float</code> oder <code>double</code> konvertiert werden
Numerische Operationen	Addition (+), Subtraktion (-), Multiplikation (*), Division (/), Modulo (%)
Ganzzahlige Division	<code>long</code> -Division verwirft Nachkommastellen (<code>5L / 2L</code> ergibt <code>2L</code>)
Lösung für echte Division	<code>double result = 5L / 2.0; // Ergebnis: 2.5</code>
Vergleichsoperatoren	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
Bitweise Operatoren	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code> , <code>>>></code>
Overflow & Underflow	<code>long</code> überläuft ohne Fehler (<code>long x = Long.MAX_VALUE + 1; //</code> <code>ergibt -9_223_372_036_854_775_808</code>)
Lösung für Overflow-Prüfung	<code>Math.addExact(long a, long b)</code> , <code>Math.multiplyExact(long a, long b)</code> werfen <code>ArithmeticException</code>
Wrapper-Methoden	<code>Long.parseLong(String s)</code> , <code>Long.valueOf(String s)</code> , <code>Long.toString(long l)</code>
Autoboxing & Unboxing	Automatische Umwandlung zwischen <code>long</code> und <code>Long</code> (<code>Long obj = 10L; long x = obj;</code>)
Konstante Werte	<code>Long.MIN_VALUE = -9_223_372_036_854_775_808</code> , <code>Long.MAX_VALUE = 9_223_372_036_854_775_807</code>
Optimierung	In vielen Fällen reicht <code>int</code> , da <code>long</code> doppelt so viel Speicher benötigt
Typischer Anwendungsfall	Verarbeitung sehr großer Zahlen (z. B. UNIX-Timestamps, große IDs, Finanz- und wissenschaftliche Berechnungen)

